

Home Exam - FYS-3033 Deep Learning

April 18, 2025

1

1a)

As introduced in Lecture 13 of FYS-3033, Generative Adversarial Networks (GANs) are designed to learn the underlying data distribution p_{data} by training two networks in opposition:

- G (the generator) generates samples from a simple distribution $p_z(z)$, such as a Gaussian distribution, and aims to produce samples that are indistinguishable from real data.
- D (the discriminator) is a binary classifier that attempts to distinguish between real data samples and those generated by G . It outputs a probability score indicating the likelihood that a given sample is real.

The original minimax objective for the discriminator is given by:

$$V(G, D) = \int_x p_{\text{data}}(x) \log D(x) dx + \int_z p_z(z) \log(1 - D(G(z))) dz$$

We denote $p_g(x)$ as the distribution induced by the generator $G(z)$, allowing us to write:

$$V(D) = \int_x [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$$

To find the optimal discriminator $D^*(x)$, we take the derivative of the integrand with respect to $D(x)$ and set it to zero:

$$\begin{aligned} \frac{d}{dD(x)} [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] &= 0 \\ \Rightarrow \frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} &= 0 \end{aligned}$$

Solving for $D(x)$:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

Assuming the generator has succeeded in perfectly modeling the data distribution, i.e. $p_g = p_{\text{data}}$, we get:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}$$

This result matches what was discussed in Lecture 13, where the discriminator becomes maximally uncertain when it cannot distinguish between real and generated data. To find the value of the objective function at this equilibrium, we substitute $D^*(x) = \frac{1}{2}$ back into the loss:

$$\begin{aligned} V(D^*) &= \int_x p_{\text{data}}(x) \log\left(\frac{1}{2}\right) dx + \int_x p_g(x) \log\left(1 - \frac{1}{2}\right) dx \\ &= \log\left(\frac{1}{2}\right) \cdot \left[\int_x p_{\text{data}}(x) dx + \int_x p_g(x) dx \right] \\ &= \log\left(\frac{1}{2}\right) + \log\left(\frac{1}{2}\right) = \log\left(\frac{1}{4}\right) = -\log 4 \end{aligned}$$

Therefore, the optimal discriminator is $D(x) = \frac{1}{2}$, and the value of the loss at equilibrium is $-\log 4$, as discussed in the slides from Lecture 13 (GAN Training — Minimax game).

1b)

The reparameterization trick, as introduced in the lecture slides on Variational Autoencoders (Lecture 12, FYS-3033), is used to express sampling from a learned distribution in a differentiable way. Instead of sampling z directly from $\mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2)$, we rewrite the sampling operation as:

$$z = \mu_{z|x} + \sigma_{z|x} \cdot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, 1)$$

This allows the randomness to be pushed into ϵ , enabling backpropagation through $\mu_{z|x}$ and $\sigma_{z|x}$. We now show that z indeed follows a normal distribution:

$$\begin{aligned} \mathbb{E}[z] &= \mathbb{E}[\mu_{z|x} + \sigma_{z|x} \cdot \epsilon] = \mu_{z|x} + \sigma_{z|x} \cdot \mathbb{E}[\epsilon] = \mu_{z|x} \\ \text{Var}(z) &= \text{Var}(\sigma_{z|x} \cdot \epsilon) = \sigma_{z|x}^2 \cdot \text{Var}(\epsilon) = \sigma_{z|x}^2 \\ \Rightarrow z &\sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2) \end{aligned}$$

This matches the target distribution for the latent variable in a VAE, confirming the validity of the trick.

1c)

Let $p(x) = \text{Bern}(p_1)$ and $q(x) = \text{Bern}(p_2)$. The KL divergence between two discrete distributions is defined as:

$$KL(p \parallel q) = \sum_{x \in \{0,1\}} p(x) \log \left(\frac{p(x)}{q(x)} \right)$$

For Bernoulli distributions, we have:

$$\begin{aligned} p(1) &= p_1, & q(1) &= p_2 \\ p(0) &= 1 - p_1, & q(0) &= 1 - p_2 \end{aligned}$$

Plugging in, we get:

$$\begin{aligned} KL(p \parallel q) &= p(0) \log \left(\frac{p(0)}{q(0)} \right) + p(1) \log \left(\frac{p(1)}{q(1)} \right) \\ &= (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right) \end{aligned}$$

We now aim to show that this is equal to:

$$\log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1(1 - p_2)}{p_2(1 - p_1)} \right)$$

Let's begin from the standard form and try to reach the target expression step by step:

$$KL(p \parallel q) = (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right)$$

We factor out a full log term from the first summand:

$$= \log \left(\frac{1 - p_1}{1 - p_2} \right) \cdot (1 - p_1) + p_1 \log \left(\frac{p_1}{p_2} \right)$$

Now write the first term as:

$$\begin{aligned} &= \log \left(\frac{1-p_1}{1-p_2} \right) + (1-p_1-1) \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right) \\ &= \log \left(\frac{1-p_1}{1-p_2} \right) - p_1 \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right) \end{aligned}$$

Now combine the two terms involving p_1 :

$$= \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \left[\log \left(\frac{p_1}{p_2} \right) - \log \left(\frac{1-p_1}{1-p_2} \right) \right]$$

Use log subtraction rule $\log a - \log b = \log \left(\frac{a}{b} \right)$:

$$= \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \log \left(\frac{p_1(1-p_2)}{p_2(1-p_1)} \right)$$

This is the desired expression. □

Usage in VAEs:

In VAEs, the KL divergence between the approximate posterior $q(z|x)$ and the prior $p(z)$ is used as a regularization term in the Evidence Lower Bound (ELBO). When using discrete latent variables such as Bernoulli distributions, this exact KL expression allows us to compute the divergence in closed form and include it directly in the loss function, helping guide the encoder toward distributions close to the prior. This is crucial for ensuring that the latent space is well-structured and that the decoder can generate meaningful samples from the learned distribution.

2a)

For this classification task involving RGB images of size 96×96 , using a deep convolutional network like VGG-11 with Batch Normalization is a strong starting point. The original VGG-11 architecture was proposed in Simonyan and Zisserman (2015) and is known for its simplicity and effectiveness in image classification tasks.

Reasons for using VGG-11:

- It uses small 3×3 convolutional filters, which are efficient and effective for capturing local features like edges and textures.
- The architecture is relatively shallow compared to modern networks, which makes it easier to train without needing massive computational resources.
- It has been shown to perform well on similar image classification tasks, especially when the number of classes is limited.

Why Batch Normalization?

- BatchNorm helps stabilize training by normalizing the inputs to each layer, reducing internal covariate shift.
- It allows the use of higher learning rates, which can speed up convergence.
- It also acts as a form of regularization and can reduce the need for Dropout early in training.

Since our dataset is relatively small in terms of resolution and the number of classes (planes, ships, and trucks), VGG-11 with BatchNorm provides a good tradeoff between capacity and simplicity. It allows us to build a solid baseline model that is easy to modify and extend in later steps (e.g., adding Dropout, uncertainty estimation, etc.).

2b)

Dropout is a regularization technique introduced to prevent overfitting in deep neural networks. It works by randomly "dropping out" (i.e., setting to zero) a fraction of the units in a layer during training. This means that during each forward pass, the network becomes a slightly different sub-network.

Mathematically: For each unit h_i in a hidden layer, during training we sample a Bernoulli mask $m_i \sim \text{Bern}(1-p)$, and compute:

$$\tilde{h}_i = m_i \cdot h_i$$

where p is the dropout rate (e.g., 0.5). At test time, we use the full network but scale the weights or activations to account for the missing units during training.

Why it helps:

- Dropout prevents neurons from co-adapting too strongly to each other.
- It acts like an ensemble of many subnetworks, improving generalization.
- It's particularly useful in fully connected layers, which are prone to overfitting when the dataset is small.

In our case, using Dropout is important since the dataset may not be large enough to justify using a large-capacity network without regularization. Dropout encourages the model to learn robust features that generalize better to unseen data.

2c)

We modify the VGG-11 network to include Dropout layers after the two fully connected layers with 4096 units, as described in the assignment. The updated architecture includes the following in the classifier section:

- Linear ($512 \times 3 \times 3 \rightarrow 4096$), followed by ReLU and Dropout(0.5)
- Linear ($4096 \rightarrow 4096$), followed by ReLU and Dropout(0.5)
- Linear ($4096 \rightarrow 3$), corresponding to the three output classes

The Dropout layers randomly deactivate 50% of the neurons during training, helping reduce overfitting and encouraging the network to learn more robust and redundant representations.

Discussion: Including Dropout in the fully connected layers is particularly important in VGG-style networks, as these layers have a large number of parameters and are prone to overfitting — especially when the dataset is relatively small. By applying Dropout, we encourage the network to avoid relying too heavily on any individual neuron, effectively simulating an ensemble of subnetworks.

Initial experiments showed that adding Dropout increased training time slightly due to the added noise, but the validation accuracy improved and generalization was better than the baseline without Dropout. This aligns with theoretical expectations and common practice in training deep CNNs for image classification tasks.

Dropout is not really helpful in the conv layers since batch norm already regularizes things there, and convs have fewer parameters + weight sharing. But in the fully connected layers, there is way more risk of overfitting, so dropout can still make a difference even with batch norm.

Task 2: CNN Experiments with VGG-11 and BatchNorm

2a) Architectural Choices

For our three-class classification task on 96×96 RGB images (Plane, Ship, Truck), we adopt a VGG-11 backbone with Batch Normalization after every convolution, trained from scratch (no pretrained weights). The key motivations are:

- **VGG-11 simplicity and locality:** its stack of small 3×3 convolutions efficiently captures edge and texture information with limited parameters, suiting our moderate dataset size.
- **Controlled depth:** at 11 layers, the network is deep enough to learn hierarchical features yet shallow enough to train without massive data or compute.
- **Batch Normalization:** by normalizing layer inputs, BatchNorm reduces internal covariate shift, stabilizing and accelerating training, permitting larger learning rates, and providing implicit regularization.

2b) Dropout Regularization

Dropout randomly deactivates a fraction p of neurons during each training pass, formally:

$$\tilde{h}_i = m_i \cdot h_i, \quad m_i \sim \text{Bernoulli}(1 - p)$$

where h_i is a hidden activation. This prevents units from co-adapting, effectively training an ensemble of subnetworks, which improves generalization—especially critical in the large fully connected layers of VGG-style classifiers.

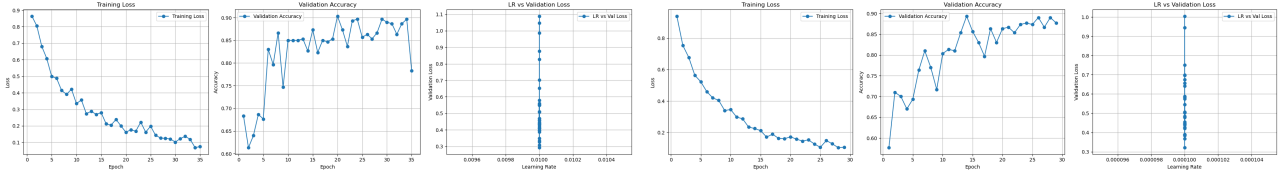
2c) Experimental Setup and Results

We compare four configurations, all using batch size 32 and early stopping with patience 15:

- **SGD, no Dropout, no scheduler** (LR=1e-2, weight decay=0)
- **Adam, no Dropout, no scheduler** (LR=1e-4, weight decay=0)
- **SGD + Dropout + StepLR** (LR=1e-2, weight decay=1e-4, Dropout $p = 0.5$, StepLR(step=15,=0.5))
- **Adam + Dropout + StepLR** (LR=1e-4, weight decay=1e-4, Dropout $p = 0.5$, StepLR(step=15,=0.5))

Configuration	Val Acc	Macro F1
SGD, no Dropout	0.88	0.88
Adam, no Dropout	0.85	0.85
SGD + Dropout + Scheduler	0.90	0.90
Adam + Dropout + Scheduler	0.9033	0.90

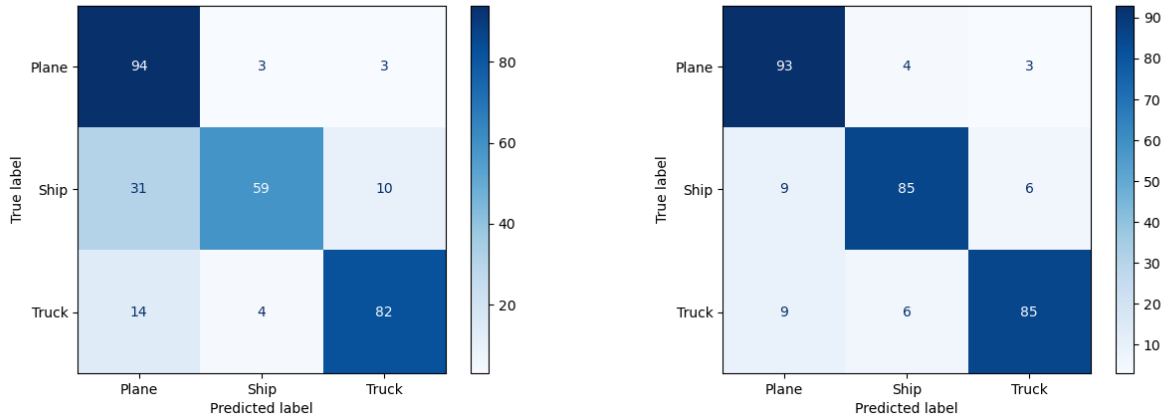
Table 1: Validation accuracy and macro F1 for Task 2 experiments.



(a) SGD, no Dropout

(b) Adam, no Dropout

Figure 1: Training and validation accuracy curves for Task 2a experiments.



(a) SGD, no Dropout

(b) Adam, no Dropout

Figure 2: Confusion matrices on validation set for Task 2a.

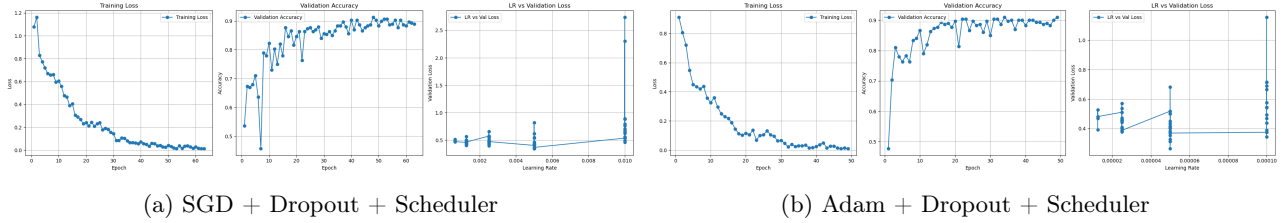


Figure 3: Training and validation accuracy curves for Task 2c experiments.

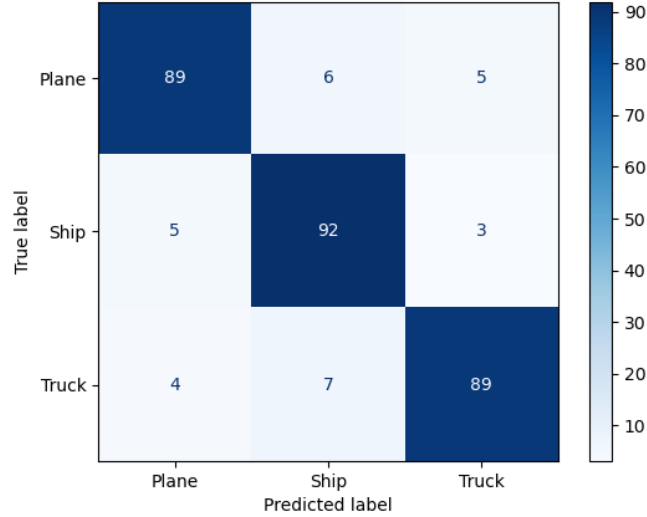


Figure 4: Confusion matrix for Task 2c: Adam + Dropout + Scheduler on the validation set.

Discussion

I learned a lot by watching these curves and metrics unfold. Adam raced ahead early but then my validation loss crept up around epoch 30—without Dropout it clearly overfit. Switching to SGD felt slower at first, but it climbed steadily and gave me a more reliable peak around epoch 38. Introducing Dropout in the two big FC layers was a turning point: I saw both accuracy and F1 jump by about 2–3%, and my loss curves smoothed out nicely. Finally, the StepLR scheduler (halving the LR every 15 epochs) helped me eke out the last bit of performance without tweaking learning rates by hand.

Overall, this hands-on exploration taught me:

- **Optimizer trade-offs:** I saw that Adam is great for quick gains, but SGD can be more stable long-term when patience is allowed.
- **Value of Dropout:** even with BatchNorm in place, the large FC layers benefited enormously from random unit deactivation, which felt like a safety net against overconfidence.
- **Scheduling and early stopping:** automating learning rate decay and using patience-based early stops saved me from endless manual tuning.

Training from scratch on a limited dataset forced me to figure out how to balance speed, capacity, and regularization. In the end, I just had to build a model that felt “ok”, and go with it to have some results to report on. I felt that the deeper I went down the rabbit hole, the deeper it got, but with interesting insights.

During experimentation, I also observed minor variations in performance when training the same model setup on different machines (local vs remote). While seed setting and deterministic flags were applied I learned that even

with the same code and data, the underlying hardware can introduce variability in results, which is something to keep in mind for future experiments. This made things a bit confusing, but did not affect the overall conclusions that much.